Ordered Parallel Composition for Tiled Splat Rendering: A Formal and CPU Reference Study

Lee Si Yuan
Independent Researcher
siyuanlee.cs@gmail.com

Abstract

Ryu and Yoo reduce the long tail in 3D Gaussian Splatting (3DGS) by scheduling heavy tiles early and splitting exceptionally heavy, depth-ordered tile lists into chunks. Each chunk yields a premultiplied color and remaining transmittance, which are merged in depth order. We study a narrow extension: represent every chunk as an affine compositing transform and combine adjacent transforms with an order-preserving balanced tree. The real-arithmetic operation is associative, so the tree preserves the exact front-to-back result while reducing the reduction dependency depth from $k-1$ to $\lceil \log_2 k \rceil$ for k chunks. A Lean 4 proof checks associativity and the identity element. A deterministic CPU reference evaluates exact rational composition and emulated float32 reassociation over 840 chunk configurations. Tree and serial composition agree exactly in all rational cases. The largest observed float32 color error against the exact result is 1.02×10^{-7}; the largest tree-versus-serial color delta is 1.79×10^{-7}. This supports algebraic feasibility and a small synthetic numerical audit only. It does not establish GPU speedup, end-to-end rendering quality, or device-specific floating-point behavior.

1 Introduction

3DGS rasterizers alpha-blend depth-sorted Gaussians independently for each screen tile (Kerbl et al., 2023). Ryu and Yoo identify a long execution tail from tile workloads that vary sharply and propose longest-processing-time scheduling, selective depth-wise chunking, and a per-frame chunking policy (Ryu and Yoo, 2026). Their chunked tile emits partial color and transmittance values that are then combined in depth order.

This note studies the reduction after chunking. We represent each contiguous chunk by its induced compositing transform and reduce the transforms with an order-preserving balanced tree. The contribution is a small algebraic and numeric study: in real arithmetic, the tree has the same output as a serial fold with logarithmic dependency depth. We make no claim that this dependency reduction translates to a faster GPU kernel.

The source paper appears in the repository's 2026-11-10 automated daily feed. A check of the paper index and paper-specific Git history found no prior citation or extension for its arXiv identifier. The feed contains metadata; the full arXiv text was consulted for this extension. This audit is limited to the repository history and available session context.

2 Ordered Compositing Monoid

For one pixel and one color channel, summarize a contiguous front-to-back interval A by $A = (C_A, T_A)$, where C_A is its premultiplied color contribution for unit incoming transmittance and T_A is its remaining transmittance. RGB composition applies the same scalar transmittance to each channel. For adjacent intervals A then B, define

$$A \odot B = (C_A + T_A C_B,\ T_A T_B). \tag{1}$$

The identity is $e = (0, 1)$. Expanding either parenthesization yields

$$\begin{aligned}(A \odot B) \odot D &= (C_A + T_A C_B + T_A T_B C_D,\\ &\qquad T_A T_B T_D),\\ A \odot (B \odot D) &= (C_A + T_A C_B + T_A T_B C_D,\\ &\qquad T_A T_B T_D).\end{aligned}$$

Thus $\odot$ is associative over the reals and has identity e. The Lean file proves these laws over $\mathbb{R}$.

For k depth-ordered chunks, a left fold performs $k-1$ pair compositions along a dependency chain. A balanced, order-preserving tree performs the same $k-1$ compositions with depth $\lceil \log_2 k \rceil$. Leaves remain in depth order and each subtree covers a contiguous interval; this is not a permutation-invariant reduction. A GPU implementation may

Measurement	Result
Exact tree = serial composition	840/840
Chunk composition = direct layer scan	all cases
Max float32 color error vs exact	1.02×10^{-7}
Max float32 transmittance error vs exact	7.25×10^{-13}
Max tree/serial float32 color delta	1.79×10^{-7}
Composition operations, either method	$k - 1$
Dependency depth, serial / tree	$k - 1$ / $\lceil \log_2 k \rceil$

Table 1: CPU reference results.

expose independent merges across lanes, warps, or CTAs, but synchronization, scratch traffic, and launch overhead could erase the theoretical benefit.

3 CPU Reference Experiment

The standard-library experiment fixes seed 20261111, generates 120 synthetic tiles, and tests chunk counts $k \in \{2, 3, 4, 8, 16, 32, 64\}$. Each tile contains 16–256 alpha layers, and alpha and color values are rational multiples of $1/100$ in $[0, 1]$. Exact fractions are used to construct chunk summaries and check the tree against both a serial chunk fold and a direct layer scan. A separate float32 emulation rounds multiplication and addition during pair composition.

Every exact rational check passes. The bounded synthetic colors make the observed float32 deviations small, but reassociation changes rounding and the maximum delta increases with the tested chunk count. These results establish the intended real-arithmetic identity for the tested generator and give a basic floating-point audit. They are not a general forward-error guarantee.

4 Formal Verification

The accompanying Lean 4 development defines a segment as a pair of real numbers and verifies associativity and left/right identity. The proof verifies the algebraic transformation that permits tree reduction. It does not verify floating-point implementation, tree construction in a renderer, image quality, or performance.

5 Limitations and Next Experiment

The available environment has no NVIDIA GPU, CUDA runtime, or CUDA compiler, and no 3DGS renderer dependencies. The CPU experiment does not model device compiler reassociation, fused multiply-add behavior, HDR or negative color values, GPU synchronization, scratch-memory traffic, occupancy, or latency. The next experiment should add the tree reduction to the source renderer, hold chunk boundaries fixed, compare against its serial reduction on real scenes, and report both reduction-kernel and end-to-end alpha-blending latency and image differences on multiple GPUs. Until then, the extension is an algebraically verified kernel hypothesis.

Reproducibility

Run `python3 experiment.py` to write `results.json`; all exact rational equality checks are assertions. The Lean proof is in `SplatCompositionClaims.lean`.

References

Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 2023. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. *ACM Transactions on Graphics*, 42(4). https://doi.org/10.1145/3592433.

Junha Ryu and Hoi-Jun Yoo. 2026. Mitigating SM-Level Load Imbalance in 3D Gaussian Splatting on GPUs. arXiv:2611.12319v1. https://arxiv.org/abs/2611.12319v1.